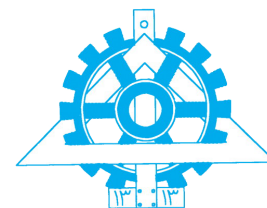




به نام خداوند رنگین کمان
طراحی کامپایلر و زبان‌های برنامه‌نویسی - بهار ۱۴۰۵
دایکومنت زبان MOL



. مقدمه و هدف پروژه

در پروژه این ترم که شامل چهار فاز اصلی می‌شود، هدف انجام مرحله به مرحله‌ی پیاده‌سازی یک کامپایلر برای زبان برنامه‌نویسی MOL است. طراحی این زبان با الهام از برخی مفاهیم زبان‌های مدرن، به ویژه زبان Rust صورت گرفته است.

. آشنایی با زبان MOL

زبان MOL یک زبان برنامه‌نویسی ماژولار است. در این زبان، کد یک برنامه در فایل‌ی با پسوند `.mol` قرار می‌گیرد؛ به عنوان مثال اگر نام فایل `test` باشد، به صورت `test.mol` ذخیره و شناسایی می‌شود. ساختار کلی یک برنامه شامل مجموعه‌ای از Top-Level Declarations است که از `module`ها و `struct`ها تشکیل می‌شود. این ساختار شباهت بسیار زیادی به مفاهیم `Class` و `Struct` در زبان `C++` دارد. نقطه شروع برنامه در این زبان ماژولی به نام `main` است که مشابه تابع `main` در زبان `C` است.

. ساختار داده‌ها: `Struct` و `Module`

بدنه اصلی برنامه‌نویسی ماژولار در این زبان بر پایه دو ساختار زیر بنا شده است:

- **Struct**: استراکت‌ها ساختارهای داده‌ای ساده‌ای هستند که صرفاً شامل فیلدها می‌شوند.
- **Module**: ماژول‌ها ساختارهای پیشرفته‌تری هستند که هم شامل فیلدها و هم شامل متدها می‌باشند.

ویژگی متمایزکننده `module` در زبان MOL این است که ماژول‌ها می‌توانند یکدیگر را توسط کلیدواژه `includes` در همان خط تعریف ماژول، شامل شوند. با این کار، یک ماژول به تمامی فیلدها و متدهای ماژول دیگر دسترسی پیدا می‌کند. این رابطه می‌تواند به صورت دوطرفه (Cyclic Inclusion) باشد؛ یعنی اگر ماژول A ماژول B را شامل شود، ماژول B هم می‌تواند ماژول A را شامل شود. همچنین لازم به ذکر است که تعریف بدنه هر `module` یا `struct` با کلیدواژه `begin` آغاز شده و با `end` خاتمه می‌یابد و فیلدها و متدها را می‌توان `public` یا `private` تعریف کرد.

. مفهوم `Explicit Mutability`

یکی از مهم‌ترین مفاهیمی که زبان MOL از زبان Rust الهام گرفته است، رویکرد ایمن در قبال مدیریت حافظه و تغییرپذیری داده‌ها است. این سیستم بر دو پایه استوار است:

● **Explicit Mutability**: در این زبان، تمامی متغیرها، فیلدها و پارامترهای ورودی متدها به صورت پیش‌فرض غیرقابل تغییر (Immutable) هستند. برای اینکه اجازه تغییر مقدار یک متغیر یا پارامتر در طول اجرای برنامه وجود داشته باشد، حتماً باید از کلیدواژه mut پیش از تعریف نوع آن استفاده کرد (به عنوان مثال `mut int x`). این ویژگی در سطح گرامر هم برای تعریف متغیرهای محلی و هم برای پارامترهای توابع اعمال شده است.

● **مالکیت (Ownership) و عدم وجود پوینتر**: در زبان MOL مفهومی به نام Reference یا Pointer وجود ندارد و در گرامر زبان هیچ نماد یا ساختاری برای ارجاع‌دهی (مانند * در C یا & در Rust) تعریف نشده است. به جای آن، زبان از مفهوم مالکیت (Ownership) و بازگشت مقادیر استفاده می‌کند. به این معنا که وقتی یک شیء به عنوان پارامتر به یک متد پاس داده می‌شود، مالکیت آن منتقل شده (Move) یا مقدار آن کپی می‌شود. از آنجایی که پوینتری وجود ندارد که دو متغیر همزمان به یک خانه از حافظه اشاره کنند، برای اعمال تغییرات روی یک شیء پیچیده درون یک تابع، آن شیء باید به عنوان خروجی متد (return) برگردانده شده و مجدداً در متغیر اولیه (که باید mut باشد) جایگذاری شود.

. انواع داده‌های پایه (Data Types)

زبان MOL یک زبان Statically Typed است. انواع داده‌ای که در این زبان پشتیبانی می‌شوند عبارتند از:

- انواع عددی: `int`، `float` و `double`
- نوع کاراکتری: `char`
- نوع منطقی: `bool`
- نوع تهی: `void` (صرفاً برای مشخص کردن نوع بازگشتی توابعی که خروجی ندارند)
- انواع تعریف شده توسط کاربر: شناسه‌ها (ID) که به نام یک `module` یا `struct` اشاره دارند، می‌توانند به عنوان نوع داده متغیرها استفاده شوند.

. ساختارهای کنترلی و Control Flow

زبان MOL برای بلاک‌بندی کدهای اجرایی، به جای استفاده از آکولاد، از کلمات کلیدی `begin` و `end` استفاده می‌کند.

- **شروط**: کنترل جریان شرطی با استفاده از کلیدواژه `if` انجام می‌شود و در صورت نیاز از `elif` و `else` پشتیبانی می‌کند.
- **حلقه‌ها**: پیاده‌سازی حلقه‌های تکرار در این زبان از طریق ساختارهای `while` و `for` انجام می‌شود.
- در این زبان همچنین `break` و `continue` با مفاهیمی مشابه زبان C وجود دارند.

. عبارات و عملگرها

گرامر زبان از طیف وسیعی از عملگرها با رعایت اولویت‌های استاندارد پشتیبانی می‌کند:

- **عملگرهای ریاضی**: جمع (+)، تفریق (-)، ضرب (*)، تقسیم (/) و باقیمانده (%).

- عملگرهای رابطه‌ای و منطقی: بررسی برابری ($=$ و $\neq$) و مقایسه بزرگی و کوچکی ($<$ ، $>$ ، $\leq$ ، $\geq$).
- عملگر انتساب: از علامت $=$ برای تخصیص مقدار به متغیرها استفاده می‌شود.
- عملگر دسترسی: برای فراخوانی متدها (Method Call) و دسترسی به فیلدهای یک شیء یا ماژول، از عملگر نقطه (.) استفاده می‌گردد.

Statement Termination .

پایان تمام دستورات باید حتماً با علامت ; خاتمه یابد تا Parser بتواند پایان یک دستور را از دستور بعدی تشخیص دهد.